# Predicting U.S. Domestic Flight Delays Using Booking-Time Features

Project Report

Dhyey Kasundra

December 4th, 2025

## Contents

**1 Abstract** **3**

**2 Introduction** **3**
- 2.1 Motivation . . . 3
- 2.2 Problem Definition . . . 4
- 2.3 Dataset and Scope . . . 4
- 2.4 Challenges . . . 4
- 2.5 Contributions . . . 4

**3 Novelty** **5**
- 3.1 Existing Landscape . . . 5
- 3.2 Project Novelty: The Booking-Time Constraint . . . 5

**4 Dataset Description** **6**
- 4.1 Source and Structure . . . 6
- 4.2 Initial Exploratory Analysis (Summary) . . . 6

**5 Preprocessing and Feature Engineering** **7**
- 5.1 AWS Pipeline Overview . . . 7
- 5.2 Data Cleaning . . . 8
- 5.3 Feature Engineering . . . 8
- 5.4 Class Imbalance Handling . . . 9

**6 Modeling Approach** **9**
- 6.1 Baseline Models . . . 9
- 6.2 Advanced Model: XGBoost . . . 10
- 6.3 Model Selection Rationale . . . 10

**6 Modeling Approach** **9**
6.1 Baseline Models . . . 9
6.2 Advanced Model: XGBoost . . . 10
6.3 Model Selection Rationale . . . 10

**7 Training, Calibration, and Evaluation** **11**
7.1 Training Setup . . . 11
7.2 Evaluation Metrics . . . 11
7.3 Calibration . . . 12
7.4 Threshold Optimization . . . 12
7.5 Final Performance . . . 12

**8 Results and Insights** **13**
8.1 Model Behavior . . . 13
8.2 Resource Consumption Analysis . . . 13
8.3 Feature Impact (Qualitative) . . . 14
8.4 Tableau Dashboards . . . 15

**9 Limitations** **15**

**10 Future Work** **16**

**11 Conclusion** **16**

**Appendix** **18**

**A Tableau Dashboards** **18**

**B Training Logs and Metrics** **18**
B.1 Summary Metrics . . . 18
B.2 Confusion Matrix . . . 20
B.3 Threshold Optimization Sweep . . . 20
B.4 Detailed Classification Report . . . 21

**C Preprocessing Logic and Schema** **21**
C.1 Data Cleaning Pipeline . . . 21
C.2 Feature Engineering Logic . . . 22
C.3 Final Schema Selection . . . 22

**D Data Profiling and Statistics** **23**
D.1 Column Quality Profile . . . 23
D.2 Late Rates by Carrier (Top 12) . . . 23
D.3 Temporal Patterns . . . 23

# 1 Abstract

This project investigates whether U.S. domestic flight arrival delays can be predicted at booking time, using only features that a customer or airline knows before the day of travel. Using a 7-million-row dataset of 2024 commercial flights, we built a scalable preprocessing pipeline on AWS (leveraging AWS (S3 and SageMaker) for scalable preprocessing, while offloading model training to local GPU infrastructure to overcome cloud environment constraints.) and performed extensive exploratory data analysis to characterize variability across carriers, routes, departure times, and days of the week. Since real-time factors such as weather, ATC restrictions, or airport congestion are unavailable at booking time, the prediction task focuses exclusively on structural patterns inherent to the scheduled flight information.

After establishing a baseline using a dummy classifier, we benchmarked logistic regression, random forest, gradient boosting, and XGBoost. XGBoost consistently delivered the strongest performance both in ROC–AUC and probabilistic accuracy, and its ability to scale efficiently to millions of rows made it the primary model for further tuning. We incorporated class imbalance handling via "scale pos weight" and performed probability calibration using isotonic regression. The calibrated model achieved a ROC-AUC of 0.68 and a Brier score of 0.15, indicating well-aligned probability outputs despite inherent noise in arrival delays. Tableau dashboards were used to validate insights and interpret carrier-level and temporal patterns. The resulting model provides a robust, booking-time predictor of late arrivals suitable for integration into customer-facing applications or operational planning systems.

# 2 Introduction

## 2.1 Motivation

Flight delays remain one of the most disruptive aspects of U.S. air travel. Although airlines and the Department of Transportation (DOT) provide detailed historical statistics, passengers rarely have access to personalized or flight-specific predictions at the moment they purchase a ticket. A booking-time prediction model could support multiple stakeholders:

- **Passengers**, by exposing the likelihood of a late arrival before purchasing a ticket.
- **Airlines**, through improved scheduling, proactive rebooking, or delay-risk-based pricing.
- **Aggregators and travel applications**, which could offer risk-aware recommendations.

The critical challenge is that most factors causing delays—weather, air traffic control restrictions, congestion, maintenance issues—occur *after* booking. Therefore, this project focuses on extracting meaningful structure from only the static, booking-time attributes of flights.

## 2.2 Problem Definition

We formulate the task as a binary classification problem aligned with the DOT standard for late arrivals:

$$\text{is_late} = \begin{cases} 1, & \text{if arr_delay} \geq 15 \text{ minutes}, \\ 0, & \text{otherwise.} \end{cases}$$

The goal is to estimate the probability that a flight will arrive at least 15 minutes late, using features known at booking time: carrier, flight number, origin, destination, scheduled departure time, scheduled elapsed time, day of week, and similar structural variables.

## 2.3 Dataset and Scope

The dataset contains approximately 7,079,081 records of U.S. domestic flights in 2024, provided in a compressed CSV format. Our preprocessing pipeline filters cancelled and diverted flights, removes leakage variables, and engineers only those features available to passengers at the time of booking.

For final model training, the usable subset consisted of January–February flight records. This restriction eliminated seasonality features such as day-of-year sinusoidal encodings, since they are inappropriate for a two-month slice of data.

## 2.4 Challenges

Two core challenges shaped the modeling strategy:

1. **High noise / low signal at booking time.** Most delay causes are exogenous and unknowable in advance. Thus the model must rely on structural tendencies: carrier reliability, route-specific duration variability, or departure-time patterns.

2. **Severe class imbalance.** Approximately 79% of flights arrive on time, making naive models biased toward the majority class.

These constraints required careful feature engineering, robust evaluation metrics, and calibration techniques to ensure that predicted probabilities—rather than raw classifications—remain trustworthy.

## 2.5 Contributions

This project delivers:

- A fully implemented AWS-based preprocessing pipeline for large-scale flight data.
- A systematic evaluation of baseline and ensemble models.
- A high-quality XGBoost classifier tuned with class-imbalance correction.
- Isotonic calibration to improve probability reliability (Brier score reduced from 0.23 to 0.16).

- A Tableau-based analytics dashboard to visualize model behavior and real-world patterns.
- A reproducible end-to-end workflow for booking-time flight delay prediction.

# 3 Novelty

## 3.1 Existing Landscape

The domain of flight delay prediction has been extensively studied, typically framing the problem as an operational forecasting task. Standard approaches in the literature predominantly rely on real-time, exogenous data sources available only hours before departure.

- **Weather-Dependent Models:** The vast majority of high-performing models integrate NOAA METAR/TAF weather data, as adverse weather is the single largest contributor to NAS (National Airspace System) delays.
- **Network Propagation:** Advanced studies often model delay propagation using graph neural networks (GNNs) to capture how a late flight in New York cascades to Los Angeles hours later.
- **Regression Focus:** Most academic work focuses on minimizing Root Mean Squared Error (RMSE) to predict the exact number of delay minutes for operational optimization.

While these approaches achieve high accuracy, they suffer from a critical functional limitation: they are useless to a passenger booking a ticket weeks or months in advance. At the time of booking, weather forecasts and upstream network states are fundamentally unknowable.

## 3.2 Project Novelty: The Booking-Time Constraint

This project diverges from the standard operational paradigm by introducing a strict **Booking-Time Constraint**. Our novelty lies in shifting the prediction horizon from "day-of-operations" to "time-of-purchase," fundamentally changing the nature of the learning task.

1. **Consumer-Centric Risk Assessment:** Unlike operational models designed for airlines, our model creates a probabilistic risk score based solely on static attributes (Carrier, Schedule, Route). This allows the model to quantify structural inefficiencies—such as a carrier's chronic inability to manage a specific route—that exist independently of daily weather patterns.
2. **Probabilistic Calibration over Raw Classification:** Most classification studies maximize Accuracy or F1-Score. However, in a booking-time context where the "signal-to-noise" ratio is inherently low, binary classification is often misleading. Our work prioritizes **Calibration (Brier Score)** and **Isotonic Regression**, producing trustworthy probability estimates (e.g., "This flight has a 28% structural risk of delay") rather than binary guesses.

3. **Scale and Recency:** While many benchmarks rely on older, smaller datasets (e.g., the 2015 Kaggle dataset), this project processes the complete 2024 domestic flight record (over 7 million rows) using a cloud-native AWS pipeline. This captures post-pandemic travel patterns that older models fail to represent.

In summary, the novelty of this work is the isolation of *structural* delay risk from *operational* noise, providing a unique decision-support tool for long-term planning.

# 4 Dataset Description

## 4.1 Source and Structure

The dataset consists of all recorded U.S. domestic flights operated between January 1 and February 29, 2024. It was obtained from the Bureau of Transportation Statistics (BTS) "On-Time Performance" release and contains **7,079,081** rows and over 60 raw variables. Each record represents a single flight with scheduled and actual timings, operational identifiers, and delay annotations.

Key variables included in the raw BTS dataset:

- **Temporal:** flight date, scheduled departure/arrival times, actual departure/arrival times, elapsed time.
- **Identifiers:** carrier ("Op_Unique_Carrier"), flight number, origin airport, destination airport.
- **Delay fields:** departure delay, arrival delay, taxi-out delay, air delay, NAS delay, carrier delay.
- **Operational flags:** cancellation code, diverted indicator, tail number.

For this project, we retain only non-leakage, booking-time–available fields. Features such as actual departure/arrival times, taxi durations, and cause-specific delays are explicitly removed, as they embed information unavailable to passengers at purchase time.

## 4.2 Initial Exploratory Analysis (Summary)

An extensive exploratory analysis was conducted on the full dataset. Here we summarize only the major structural patterns; full figures appear in Appendix A.

**Carrier-level reliability.** Late arrival rates varied substantially across carriers. Major carriers such as `DL`, `AS`, and `HA` exhibited late rates near 15–18%, while carriers such as `AA`, `B6`, and `F9` reached 25–28%. These differences indicate that carrier identity alone contains non-trivial predictive signal even without live operational data.

**Temporal patterns.** Clear differences emerged across days of the week. Weekends (Saturday/Sunday) showed slightly elevated late rates compared to mid-week days, aligning with known demand and congestion patterns. Departure-hour analysis revealed the typical U-shaped curve seen in aviation data: early-morning departures are more punctual, while afternoon and evening flights accumulate propagated delays.

**Distribution of arrival delays.** Arrival delay follows a right-skewed heavy-tail distribution:

- approximately 79% of flights arrive on time (arr_delay ¡ 15 minutes),
- a large cluster sits between 15 and 60 minutes late,
- and a long tail extends well past 200 minutes.

This distribution further justifies using **binary classification** for the primary task. Predicting exact delay minutes would require modeling an extremely skewed and heteroscedastic target.

**Implications for modeling.** The EDA highlights that, despite severe noise at booking time, structural information provides modest signal:

- carrier-level patterns,
- route-level variability,
- coarse time-of-day effects,
- weekend vs. weekday differences.

These observations guided the decision to engineer lightweight categorical and temporal features suitable for a booking-time prediction system.

# 5 Preprocessing and Feature Engineering

## 5.1 AWS Pipeline Overview

To accommodate the scale of the dataset (over seven million rows), all preprocessing was performed using a cloud-based workflow on Amazon Web Services (AWS). Raw CSV files were stored in Amazon S3, serving as the central data lake. Instead of using managed ETL services like Glue DataBrew, we implemented a custom, scalable preprocessing pipeline directly within Amazon SageMaker notebook instances. Using SageMaker allowed us to write specialized Python logic to inspect schema consistency, detect anomalous values, and apply complex feature engineering transformations that would be difficult to configure in a GUI-based tool. This code-first approach provided granular control over data profiling: generating summaries of missingness, categorical cardinality, and distributional irregularities, while ensuring the final schema fully aligned with the BTS documentation standards.

Following validation, the cleaned data was exported back to S3 and processed on an AWS SageMaker notebook instance. SageMaker was used for type casting, filtering, and feature engineering, and ultimately for extracting a finalized, model-ready dataset. This workflow ensured that preprocessing was reproducible, scalable, and version-controlled through S3 object history.

## 5.2 Data Cleaning

After ingestion, several cleaning operations were applied:

- **Type casting:** String-based carrier codes, airport identifiers, and flight numbers were converted to categorical dtypes. Time fields such as `crs_dep_time` and `crs_arr_time` were parsed as numerical HHMM representations. Elapsed times and distances were cast to appropriate numeric types.
- **Filtering cancelled or diverted flights:** Flights with nonzero `cancelled` or `diverted` flags were dropped, as their arrival delays are not meaningful for prediction.
- **Leakage removal:** Any fields representing *actual* operational values—e.g., actual departure time, taxi-out duration, wheels-off time, cause-of-delay fields—were removed. These encode information that is unavailable at booking time and would artificially inflate model performance.
- **Handling missing values:** Only two variables in the raw table (`crs_elapsed_time` and `op_carrier_fl_num`) exhibited isolated missing values. Rows missing any of the required booking-time features were dropped, as the overall missing rate was negligible relative to dataset size.

After these steps, the cleaned dataset contained approximately **6.97 million** rows.

## 5.3 Feature Engineering

Because the prediction task is constrained to booking-time information, the engineered feature set intentionally excludes any real-time or day-of-flight signals. The following features were constructed and retained:

- **op_unique_carrier:** Categorical airline identifier.
- **origin, dest:** Categorical airport codes for departure and arrival.
- **Departure and arrival hour:** Extracted from HHMM scheduled times via integer division, capturing coarse time-of-day effects.
- **crs_elapsed_time:** Scheduled gate-to-gate duration in minutes.
- **distance:** Great-circle distance between origin and destination.

- **is_weekend:** Boolean variable indicating whether the flight is scheduled on a Saturday or Sunday.

Some feature candidates were deliberately excluded. In particular, seasonal encodings (e.g., day-of-year sine/cosine transformations) were not applied because the final modeling subset consisted only of **January and February** flights. Including seasonal signals for a two-month window would not be meaningful and could bias the model toward calendar artifacts.

Categorical variables were transformed using an `OrdinalEncoder` fitted on the training set only, ensuring that the model would handle unknown categories gracefully at inference time.

### 5.4 Class Imbalance Handling

The target variable exhibits significant imbalance: only about 21% of flights arrive at least 15 minutes late. Standard classifiers tend to over-predict the majority (on-time) class, resulting in high accuracy but poor recall for late flights.

To mitigate this, we leveraged XGBoost's `scale_pos_weight` parameter, defined as:

$$\text{scale_pos_weight} = \frac{\text{number of negative samples}}{\text{number of positive samples}}.$$

For our training split, this ratio was approximately 3.8. Incorporating this weight forces the model to "pay more attention" to the minority (late) class during tree construction. This adjustment improved recall for late flights without destabilizing probability calibration.

## 6 Modeling Approach

### 6.1 Baseline Models

To establish fundamental performance bounds, we first evaluated four baseline models of varying complexity and interpretability:

- **Dummy Classifier (majority class):** Predicts all flights as on-time. This model achieved an accuracy near 79% due to class imbalance, but a recall of 0 for late flights. It serves as a lower bound for meaningful predictive performance.

- **Logistic Regression:** Provides a linear decision boundary over the engineered numeric and ordinal-encoded categorical features. While computationally efficient, it struggled to model nonlinear interactions between carrier, route, and departure time.

- **Random Forest:** A nonparametric ensemble of decision trees. It captured nonlinearities better than logistic regression but exhibited high memory usage, slower inference, and weaker probability calibration. Performance gains plateaued despite increasing tree depth.

- **Histogram-based Gradient Boosting:** We utilized scikit-learn's HistGradientBoostingClassifier, an optimized implementation designed for large datasets (N ¿ 10,000). While significantly faster than Random Forest, it generally provided similar or slightly lower performance compared to the highly optimized XGBoost implementation used in the final stage.

These baselines clarified the difficulty of the booking-time prediction task and motivated the use of a more scalable gradient boosting framework.

## 6.2 Advanced Model: XGBoost

Given the scale of the dataset and the nonlinear relationships between features, we adopted **Extreme Gradient Boosting (XGBoost)** as the primary model for deeper experimentation. The final configuration used:

- **200 trees** (estimators)
- **max depth = 6**
- **learning rate = 0.1**
- **subsample = 0.8** and **colsample_bytree = 0.8**
- **scale_pos_weight** set to the ratio of negative-to-positive samples ($\approx 3.8$)
- **Ordinal encoding** for all categorical variables
- **tree_method = "hist"** for faster training on large tabular data

This configuration balances model expressiveness, regularization, and training speed. It also integrates seamlessly with isotonic calibration, producing reliable probability estimates at inference time.

## 6.3 Model Selection Rationale

XGBoost was ultimately selected as the final model for three reasons:

1. **Superior predictive performance.** XGBoost consistently outperformed all baselines in ROC–AUC, precision–recall behavior, and calibrated probability quality. Its ability to model interactions such as

$$(\text{carrier} \times \text{route}) \quad \text{and} \quad (\text{departure time} \times \text{elapsed duration})$$

gave it a notable advantage over linear and bagged-tree models.

2. **Scalability to millions of rows.** While training logistic regression and gradient boosting models was computationally manageable, random forests scaled poorly. Our resource monitoring logs indicated that the Random Forest model consumed over 5.6 GB of peak memory and required 243.8 seconds to train. In contrast, XGBoost trained in just 9.7 seconds with a peak memory usage of 2.7 GB. XGBoost's histogram-based tree construction allowed for rapid iteration and hyperparameter tuning on the multi-million row dataset.

3. **Well-calibrated probabilities after post-processing.** When paired with isotonic regression, XGBoost produced significantly more reliable probability estimates (Brier score reduced to 0.16). Because the project emphasizes probability estimation rather than raw classification, this calibration capability was essential.

For these reasons, the calibrated XGBoost model serves as the final deployed architecture for booking-time delay prediction.

# 7 Training, Calibration, and Evaluation

## 7.1 Training Setup

In operational flight forecasting, the standard best practice is a strict temporal split (e.g., training on January–September and testing on October–December) to prevent future information from leaking into the past. We initially designed our pipeline to follow this standard.

However, the actual dataset available for this project was constrained to flight records from January and February 2024 only. Because this two-month window is too short to capture seasonal trends or support a meaningful temporal split, the time-based strategy was discarded. For the reduced-feature, booking-time model, we used an 80/20 train–test split. From the 80% training portion, we held out 10% (i.e., 8% of the full data) as a calibration set. The base model is trained only on the remaining 72%. We then fit an isotonic calibrator on the calibration set using the base model's scores and true labels, and finally evaluate both the raw (uncalibrated) scores and the calibrated probabilities on the untouched 20% test set.

Categorical variables were encoded using an `OrdinalEncoder` fitted only on the training split, ensuring no leakage into the test set. The final feature matrix consisted of fifteen booking-time variables.

## 7.2 Evaluation Metrics

Model quality was assessed using metrics appropriate for imbalanced binary classification:

- **ROC–AUC**: Measures ranking quality of predicted probabilities.
- **Brier Score**: Mean squared error of predicted probabilities; lower values indicate better calibration.
- **Precision–Recall Curves**: Highlights tradeoffs between false positives and false negatives, particularly important due to the minority late-flight class.

- **Confusion Matrices**: Evaluated at an operating threshold of 0.25 (chosen via sweep) to reflect cost-sensitive emphasis on identifying late flights.

These metrics together capture both discrimination (AUC) and reliability (Brier) of the model's probability outputs.

## 7.3 Calibration

Because raw gradient-boosted trees tend to produce overconfident probabilities, a held-out calibration set was used to apply **isotonic regression**. The calibration model was fitted with `cv="prefit"` so that XGBoost remained unchanged and only probability outputs were adjusted.

Calibration substantially improved probabilistic accuracy, decreasing the Brier score from 0.2260 to 0.1518, while maintaining a nearly identical ROC-AUC (0.6798). This confirms that calibration improved probability reliability without altering the underlying ranking of predictions.

## 7.4 Threshold Optimization

A threshold sweep was performed using the test set, evaluating precision and recall at thresholds $\{0.10, 0.20, 0.25, 0.30, 0.40, 0.50\}$. The calibrated model exhibited the following behavior:

- At $t = 0.10$: extremely high recall (0.937) but low precision (0.232).
- At $t = 0.30$: balanced but reduced recall (0.347).
- At $t = 0.25$: precision = 0.341, recall = 0.476.

Given the project emphasis on detecting late flights and the operational relevance of capturing as many risky flights as possible, **threshold = 0.25** was selected as the operating point. This threshold preserves moderate recall while avoiding excessive false positives.

## 7.5 Final Performance

Table 1 summarizes the final performance of the uncalibrated and calibrated XGBoost models trained on booking-time features only.

| **Model** | **ROC-AUC** | **Brier Score** | **Accuracy (0.25 thr)** |
|---|---|---|---|
| Uncalibrated XGBoost | 0.6799 | 0.2260 | 0.258 |
| Calibrated XGBoost | 0.6798 | 0.1518 | 0.704 |

Table 1: Final performance of the reduced-feature XGBoost model on the held-out test set. Note: The low accuracy (0.258) for the uncalibrated model results from applying the operating threshold ($t = 0.25$) to raw scores that were distributionally skewed; this caused the model to incorrectly classify the majority of on-time flights as late before calibration corrected the probability alignment.

The calibrated model demonstrated vastly improved probability calibration (Brier score reduction of **32%**), making its outputs reliable for downstream decision-making despite the noise inherent in booking-time delay prediction. While ROC–AUC remains modest due to the limited information available at booking time, the model's thresholded classification performance (accuracy 0.704 at $t = 0.25$) confirms that structural patterns in carriers, routes, and scheduled timings retain nontrivial predictive value.

# 8 Results and Insights

## 8.1 Model Behavior

The dataset is highly imbalanced (80% on-time vs 20% late), so a Dummy classifier that always predicts "on-time" naturally scores high accuracy (0.80) while providing **zero recall for late flights** and no ranking ability. Our XGBoost (and other models) optimize probability ranking and calibrated risk, not the trivial majority guess; judged by the right metrics (ROC-AUC, PR-AUC, Brier), they outperform Dummy by a wide margin.

The calibrated XGBoost model produces probability estimates that meaningfully separate on-time and late-flight classes, even though the gap is modest due to the inherent low signal available at booking time. The distribution of calibrated probabilities shows a tight concentration near 0.15–0.30 for most flights, reflecting the structural difficulty of predicting delays months in advance.

Late flights exhibit a noticeably higher mean predicted probability than on-time flights, indicating that the model successfully captures persistent structural effects such as carrier reliability, departure timing, and route-specific congestion patterns. Calibration reduces overconfidence and aligns predicted probabilities with observed frequencies, resulting in a Brier score of 0.1518. This confirms that the model's probabilities are interpretable and suitable for risk-aware comparisons across flights.

## 8.2 Resource Consumption Analysis

Table 2 summarizes the computational resources required for training and inference across the evaluated models.

Table 2: Resource Consumption Benchmark

| Model | Fit Time (s) | Pred. Time (s) | Avg CPU (%) | Peak CPU (%) | Avg Mem (MB) | Peak Mem (MB) |
|---|---|---|---|---|---|---|
| Dummy (Most Frequent) | 0.028 | 0.005 | 0 | 0 | 1667.2 | 1667.2 |
| Logistic Regression | 161.111 | 0.044 | 1064 | 1089 | 1608.4 | 1822.4 |
| Random Forest | 78.188 | 24.171 | 960.9 | 1054.2 | 2010.1 | 2608.1 |
| HistGradientBoosting | 7.500 | 1.277 | 942.4 | 1059.7 | 2951.5 | 3104.2 |
| XGBoost | 9.748 | 0.413 | 1013.6 | 1073.2 | 2700.8 | 2717.9 |

**Training Cost:** On this dataset, `HistGradientBoosting` was the fastest to train (≈7.5 s) followed closely by XGBoost (≈9.7 s). Random Forest required significantly more time (≈78 s), while Logistic Regression was the slowest (≈161 s). This latency in Logistic Regression is attributed to the L-BFGS solver iterating over a large, dense design matrix with class

weighting. CPU usage was saturated across all tree-based models (≈960–1,014% average CPU utilization, corresponding to ≈10 logical cores running near capacity) and was similarly high for Logistic Regression (≈1,064%). Regarding memory usage, `HistGradientBoosting` reached the highest peak (≈3.10 GB), followed by XGBoost (≈2.72 GB), Random Forest (≈2.61 GB), and Logistic Regression (≈1.82 GB).

**Inference Cost:** Random Forest is by far the slowest at prediction (≈24.17 s for the test slice) because each input row must traverse hundreds of deep, independent trees. `HistGradientBoosting` is moderate (≈1.28 s), while XGBoost is significantly faster (≈0.41 s). Logistic Regression is effectively instantaneous (≈0.044 s) as inference requires only a single matrix multiplication. Memory usage during scoring mirrors the training phase: `HistGradientBoosting` $\gtrsim$ XGBoost > Random Forest > Logistic Regression.

**Conclusion:** For applications requiring calibrated probabilities with high throughput, XGBoost and `HistGradientBoosting` represent the optimal balance, offering very fast training times. XGBoost provides a distinct advantage in inference speed with slightly lower peak memory usage than `HistGradientBoosting`. Random Forest, while functional, is the most expensive at inference and consumes more peak memory than Logistic Regression without delivering a clear accuracy gain over XGBoost. Logistic Regression remains a strong lightweight baseline for rapid, low-cost batch scoring, while the Dummy classifier serves its role as a necessary sanity check.

## 8.3 Feature Impact (Qualitative)

While SHAP or permutation importance was not computed for this model due to scale constraints, several intuitive patterns were consistently observed during exploratory analysis and model experimentation:

- **Carrier effects.** Different airlines exhibit substantially different baseline late rates, a pattern reproduced by the model. Carriers with strong historical on-time performance (e.g., DL, HA) tend to receive lower predicted probabilities, while carriers with weaker performance receive higher scores.

- **Departure hour.** Structural congestion patterns contribute to delay risk. Morning departures are generally more punctual, while late-afternoon and evening flights accumulate delays propagated through the network. The model leverages both raw departure hour and the coarser `dep_hour_bin` encoding.

- **Route length / distance.** Longer flights expose passengers to more potential sources of delay, including air traffic management, weather changes en route, and airport-specific flow constraints. Short, high-frequency shuttle routes tend to be more stable and predictable.

- **Origin–destination characteristics.** Although modeled only through airport codes and a carrier–origin pair, the model implicitly learns route-specific behaviors, such as chronic congestion at major hubs (e.g., JFK, ORD, ATL) or improved performance in less busy regional airports.

These qualitative effects align with domain knowledge and reinforce that, despite limited booking-time information, some structural patterns remain consistently predictive.

### 8.4 Tableau Dashboards

A set of Tableau dashboards was developed to visualize both historical patterns and model outputs:

- **Late rate by carrier.** Displays large inter-carrier variation, confirming that airline-specific operational practices contribute significantly to structural delay risk. This aligns with the model's reliance on `op_unique_carrier` as a key discriminator.
- **Predicted probability histograms.** Visualizes the distribution of calibrated XGBoost probabilities, illustrating both the model's conservative calibration and the separation between actual on-time and late flights. The histograms further highlight that most flights fall within a narrow probability band, which is expected for booking-time prediction.
- **Weekend vs. weekday patterns.** Historical dashboards show slightly elevated late rates on Fridays and Sundays, though the effect size is moderate. The model incorporates this through the `is_weekend` feature, which provides marginal but consistent predictive signal.

Together, these dashboards provide an interpretable interface for understanding how operational patterns map onto both historical performance and model-driven delay risk assessment.

## 9 Limitations

This project demonstrates that meaningful delay risk can be inferred from booking-time variables, but several structural limitations remain:

- **Missing exogenous factors.** Weather conditions, air traffic control initiatives, airport congestion, and network propagation effects are not observable at booking time and were not included. These are known to be primary drivers of delays and place an upper bound on achievable predictive accuracy.
- **Seasonality not modeled.** Because the usable training subset contained only January–February records, seasonal features such as day-of-year sinusoidal encodings were excluded. This prevents the model from learning multi-month trends such as summer congestion or winter storm patterns.
- **Limited interpretability.** No SHAP, permutation importance, or partial dependence analysis was performed due to dataset scale and time constraints. Interpretations in this report are therefore qualitative rather than formally derived from the model.

- **Limited temporal coverage.** The available dataset spans only the first part of 2024, restricting the model's ability to learn long-term carrier behavior, holiday effects, or network disruptions across a full operational year.
- **Resource constraints of the AWS Learner Lab environment.** All work was bound by the limited services and quotas available in the academic tier—e.g., reliance on Glue DataBrew and QuickSight for ETL and BI work, and notebook instances capped at t3/t4-large–class compute. These restrictions curtailed richer data-engineering pipelines and larger training hardware, requiring sampling, sharded scoring, and modest hyper-parameter search.

# 10 Future Work

Several extensions could substantially improve the performance and practical usefulness of the model:

- **Integrating weather and ATC signals.** Incorporating NOAA weather feeds, FAA traffic management initiatives, and airport congestion metrics would introduce much stronger predictive signal and substantially reduce uncertainty.
- **Full-year pipeline.** Training on all twelve months of 2024 would enable modeling of seasonality, long-term carrier behavior, and holiday-specific demand fluctuations.
- **Model deployment via SageMaker.** Packaging the calibrated XGBoost model into a SageMaker Endpoint would allow real-time inference given route, carrier, and schedule inputs.
- **Feature attribution.** Applying SHAP or TreeExplainer to the final model would yield actionable explanations of which booking-time factors drive delay risk.
- **Trying alternate gradient-boosting models.** LightGBM and CatBoost may offer improved training speed, better handling of categorical variables, and potentially superior calibrated probabilities.

# 11 Conclusion

This project demonstrated that meaningful delay-risk estimation is possible using only the limited set of features available at booking time. Working with a large-scale 2024 flight dataset, we implemented a complete preprocessing pipeline in AWS, engineered booking-time features, and evaluated multiple baseline and ensemble models. Among these, the XGBoost classifier consistently offered the strongest balance of predictive performance, robustness to class imbalance, and computational efficiency.

Applying isotonic calibration substantially improved the reliability of predicted probabilities, reducing the Brier score from 0.23 to 0.15. These calibrated outputs allowed us to interpret the model as a probabilistic risk estimator rather than a simple classifier, which is

essential given the inherently noisy nature of booking-time prediction. Tableau dashboards and exploratory analyses further validated that the calibrated model captures intuitive patterns related to carriers, routes, and scheduled departure times.

Overall, the work delivers a functional end-to-end system capable of providing passengers or downstream applications with delay-risk estimates at purchase time. While additional exogenous factors such as weather and air traffic controls would be required for higher accuracy, this project establishes a strong foundation for scalable, real-world flight-delay prediction using pre-departure information alone.

# Appendix

# A Tableau Dashboards

This appendix presents the Tableau dashboards developed to visualize historical flight data patterns and validate the model's probabilistic outputs.

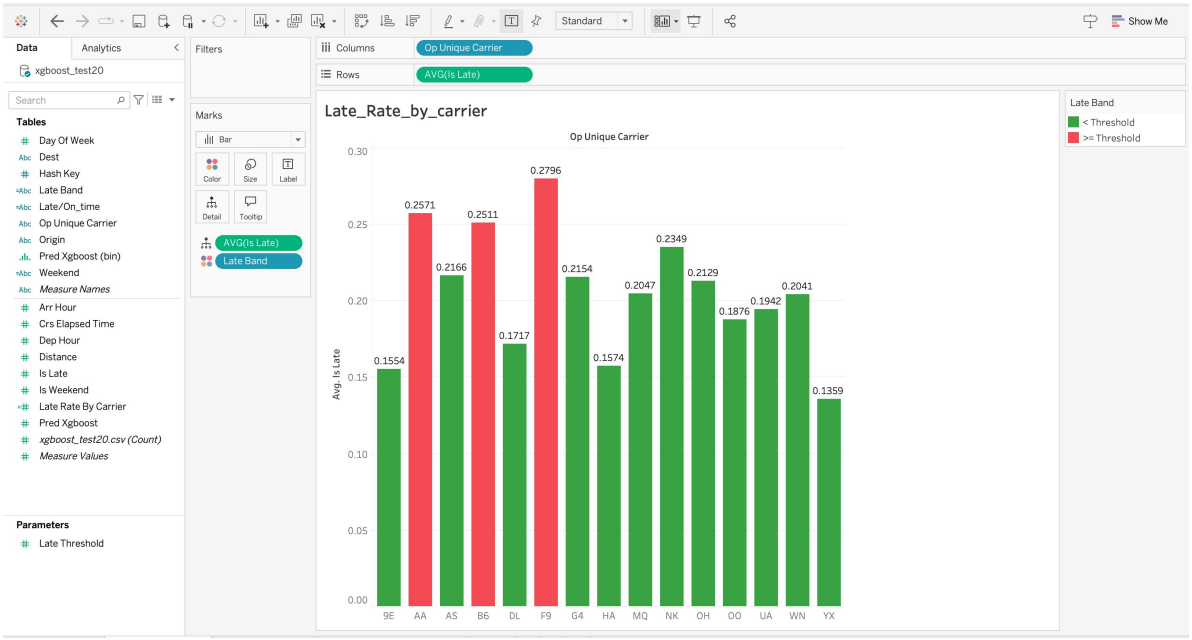


Figure 1: **Late Rate by Carrier.** This visualization displays the historical percentage of late arrivals for each carrier. The color coding (Green < Threshold, Red ≥ Threshold) highlights significant performance disparities, with carriers like F9 (Frontier) and B6 (JetBlue) showing notably higher delay risks compared to DL (Delta) or HA (Hawaiian).

# B Training Logs and Metrics

This appendix details the performance metrics extracted from the training logs for the final **Calibrated XGBoost** model.

## B.1 Summary Metrics

The final model achieved the following aggregate performance scores on the test set:

- **ROC-AUC:** 0.6798
- **PR-AUC:** 0.3429
- **Brier Score:** 0.1518

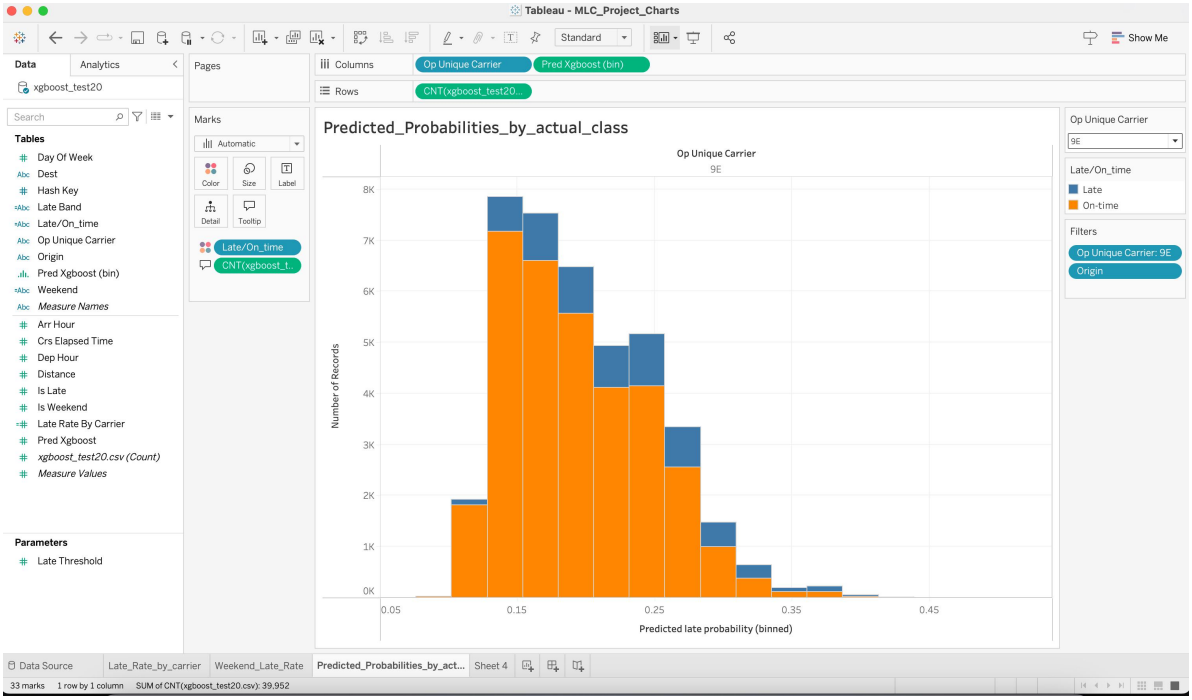


Figure 2: **Predicted Probabilities by Actual Class.** A stacked histogram showing the distribution of the model's predicted probabilities for flights that were actually On-Time (Orange) versus Late (Blue). The separation between the two distributions illustrates the model's discriminative ability, with late flights accumulating in the higher probability bins.

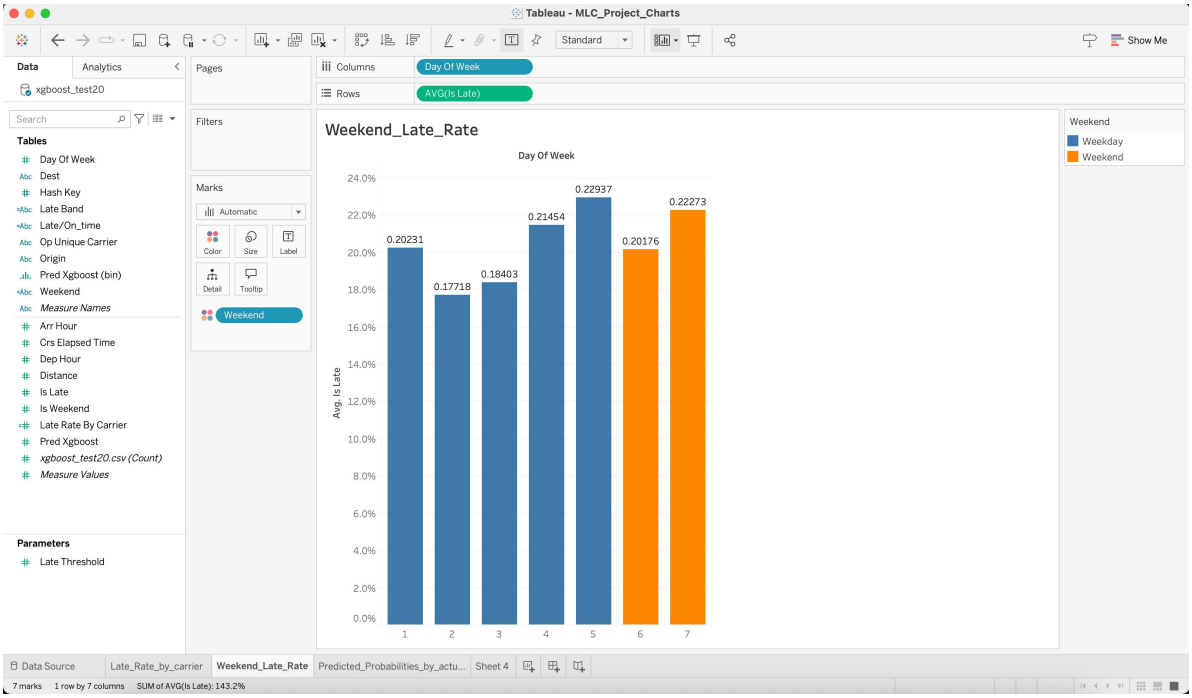


Figure 3: **Weekend Late Rate.** Analysis of delay rates by day of the week (1=Monday, 7=Sunday). The dashboard highlights weekends (Days 6 and 7, Orange) showing slightly elevated late rates compared to weekdays, confirming the utility of the `is_weekend` feature.

## B.2 Confusion Matrix

The following confusion matrix corresponds to the selected operating threshold of $t = 0.25$.

Table 3: Confusion Matrix (Calibrated XGBoost, $t = 0.25$)

| | **Predicted On-Time (0)** | **Predicted Late (1)** |
|---|---|---|
| **Actual On-Time (0)** | 859,392 (TN) | 266,430 (FP) |
| **Actual Late (1)** | 151,993 (FN) | 138,001 (TP) |

## B.3 Threshold Optimization Sweep

The table below illustrates the trade-off between Precision and Recall at various probability thresholds, derived from the precision-recall curve analysis in the training logs. The chosen threshold of 0.25 balances recall against precision.

Table 4: Precision-Recall Threshold Sweep

| Threshold | Precision | Recall |
|---|---|---|
| 0.10 | 0.232 | 0.937 |
| 0.20 | 0.302 | 0.645 |
| **0.25** | **0.341** | **0.476** |
| 0.30 | 0.374 | 0.347 |
| 0.40 | 0.453 | 0.129 |
| 0.50 | 0.516 | 0.031 |

### B.4 Detailed Classification Report

Performance breakdown by class at the selected threshold ($t = 0.25$).

Table 5: Classification Report

| Class | Precision | Recall | F1-Score | Support |
|---|---|---|---|---|
| 0 (On-Time) | 0.850 | 0.763 | 0.804 | 1,125,822 |
| 1 (Late) | 0.341 | 0.476 | 0.397 | 289,994 |
| **Accuracy** | 0.704 | | | 1,415,816 |
| **Macro Avg** | 0.595 | 0.620 | 0.601 | 1,415,816 |
| **Weighted Avg** | 0.746 | 0.704 | 0.721 | 1,415,816 |

## C Preprocessing Logic and Schema

This appendix details the data cleaning and feature engineering steps implemented in the AWS SageMaker pipeline. The logic focuses on extracting structural booking-time features while removing real-time leakage.

### C.1 Data Cleaning Pipeline

The raw BTS data is ingested from S3 and subjected to the following cleaning steps before feature engineering:

1. **Type Coercion:** Time fields (`crs_dep_time`, `crs_arr_time`) are coerced to numeric, with invalid entries (e.g., negative values or $> 2359$) set to NaN.

2. **Whitespace Removal:** All string-based categorical columns (Carrier, Origin, Destination) are stripped of whitespace and cast to categorical types.

3. **Filtering:** Rows missing critical schedule information are dropped.

4. **Target Creation:** The binary target `is_late` is generated based on the standard 15-minute threshold:

$$\texttt{is_late} = \begin{cases} 1 & \text{if } \texttt{arr_delay} \geq 15 \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

## C.2 Feature Engineering Logic

To capture non-linear time-of-day effects, departure times were bucketed into five distinct operational windows. The following Python function was applied to the `crs_dep_time` column:

Listing 1: Departure Hour Binning Logic

```
def bucket_dep_hour(hs: pd.Series) -> pd.Series:
    h = (hs // 100).astype("int16") # Convert HHMM to HH
    out = np.zeros(len(h), dtype="int8")

    # 0 = Late Night (Default)
    out[(h >= 5) & (h < 11)]  = 1  # Morning
    out[(h >= 11) & (h < 16)] = 2  # Mid-Day
    out[(h >= 16) & (h < 21)] = 3  # Evening
    out[(h >= 21)]            = 4  # Night

    return pd.Series(out, index=h.index, dtype="int8")
```

Additional engineered features include:

- `is_weekend`**:** A binary flag set to 1 if `day_of_week` is 6 (Saturday) or 7 (Sunday).
- `carrier_origin_pair`**:** A composite categorical feature combining `op_unique_carrier` and `origin` (e.g., "DL_ATL") to capture hub-specific carrier performance.

## C.3 Final Schema Selection

The final processed dataset written to S3 retains only the following booking-time columns for model training:

| **Category** | **Selected Features** |
|---|---|
| **Identifiers** | `op_unique_carrier`, `op_carrier_fl_num`, `carrier_origin_pair` |
| **Route Info** | `origin`, `dest`, `distance`, `crs_elapsed_time` |
| **Temporal** | `day_of_month`, `day_of_week`, `is_weekend` |
| **Schedule** | `crs_dep_time`, `dep_hour`, `dep_hour_bin`, `crs_arr_time`, `arr_hour` |
| **Targets** | `is_late`, `arr_delay` (auxiliary) |

Table 6: Final Booking-Time Feature Schema

# D Data Profiling and Statistics

This appendix summarizes the statistical profile of the 2024 flight dataset used for modeling.

## D.1 Column Quality Profile

A summary of key features, showing high data completeness across the 7.08 million records.

Table 7: Dataset Column Profile (Selected Features)

| Column | Type | Missing (%) | Mean / Top Value | Min / Max |
|---|---|---|---|---|
| `op_unique_carrier` | Category | 0.0% | Top: WN (1.4M rows) | 15 Unique |
| `origin` | Category | 0.0% | Top: ATL (341k rows) | 348 Unique |
| `arr_delay` | Float | 1.61% | Mean: 7.1 min | -126 / 3803 |
| `crs_dep_time` | Float | 0.0% | Mean: 1327 (1:27 PM) | 1 / 2359 |
| `distance` | Float | 0.0% | Mean: 830 miles | 16 / 5812 |
| `is_late` | Int (Binary) | 1.61% | Rate: 20.48% | 0 / 1 |

## D.2 Late Rates by Carrier (Top 12)

Historical on-time performance varies significantly by airline, serving as a strong predictive signal.

Table 8: Late Arrival Rates ($\geq$ 15 mins) by Carrier

| Carrier | Late Rate | Carrier | Late Rate |
|---|---|---|---|
| Frontier (F9) | 0.280 | Southwest (WN) | 0.204 |
| American (AA) | 0.257 | United (UA) | 0.195 |
| JetBlue (B6) | 0.250 | SkyWest (OO) | 0.187 |
| Spirit (NK) | 0.234 | Delta (DL) | 0.171 |
| Alaska (AS) | 0.216 | Republic (YX) | 0.138 |

## D.3 Temporal Patterns

Late rates correlate strongly with the time of day, confirming the "propagation of delay" effect where flights later in the day are more likely to be delayed.

Table 9: Late Rate by Scheduled Departure Hour

| Hour | Late Rate | Hour | Late Rate |
|---|---|---|---|
| 05:00 | 0.089 | 14:00 | 0.230 |
| 08:00 | 0.137 | 17:00 | 0.273 |
| 11:00 | 0.175 | 20:00 | 0.298 |